

Team OKRs: Django + Infrastructure + Celery

Quarter: Q1 2026

Team Size: Backend Engineering Team

Focus Areas: API Performance, Background Processing, Infrastructure Efficiency

TEAM OKRs

Shared accountability across all team members

Objective 1: Improve API Reliability and Request Performance

Key Results:

1. Reduce P95 API response time from 2000ms → 300ms
2. Reduce production errors (5xx) by 35% quarter-over-quarter
3. Achieve 99.9% uptime for all public APIs
4. Reduce database query count per request by 20% on top 5 endpoints

Applies to: Django views, serializers, ORM usage, middleware, caching

Objective 2: Make Background Processing Predictable and Fast

Key Results:

1. Reduce average Celery task execution time by 30%
2. Keep Celery queue backlog under 2 minutes at peak (95% of the time)
3. Reduce task retry rate to <5%
4. Eliminate all unbounded or long-running tasks (>10 minutes)

Applies to: Task design, retries, idempotency, queue routing

Objective 3: Reduce Operational Risk and Firefighting

Key Results:

1. Reduce implementations related failures that arise in production workloads by 100%
2. 100% of P1/P2 incidents have postmortems within 72 hours, and these must be relayed by the developer with the last PR on said path
3. Add health checks + alerts for all Celery queues

INDIVIDUAL OKRs

Role-based objectives that ladder up to team goals

Backend Engineer (Django-focused)

Objective: Make Django APIs fast, predictable, and maintainable

Key Results:

1. Optimize top 3 slowest endpoints ($\geq 25\%$ speed improvement each)
2. Reduce N+1 queries by 90% in owned services
3. Add caching (Redis/DB) to 5 high-traffic endpoints
4. Write performance tests for all new critical endpoints

Celery / Async (Django-focused)

Objective: Make background jobs fast, observable, and safe

Key Results:

1. Split long-running tasks into idempotent sub-tasks
2. Achieve visibility on task success rates.
3. Reduce task failure rate to $< 2\%$
4. Add metrics for execution time.

Infrastructure / DevOps Engineer

Objective: Ensure Django & Celery scale reliably under load

Key Results:

1. Implement autoscaling rules for web & workers based on CPU + queue depth
2. Reduce infrastructure cost waste by 20% (rightsizing, idle resources)
3. Improve deployment rollback time to <5 minutes
4. Ensure zero manual SSH fixes in production

Database / Performance Owner

Objective: Keep the database fast under growing load

Key Results:

1. Reduce slow queries (>500ms) by 40%
2. Add missing indexes for top 10 heavy queries
3. Keep DB CPU utilization under 70% at peak
4. Implement query monitoring + alerts

Tech Lead / Engineering Manager

Objective: Maintain velocity without creating operational debt

Key Results:

1. Reduce hotfix deployments by 30%
2. Create visibility on hot-paths
3. Enforce performance review on 50% of PRs touching hot paths
4. Ensure every service has a clear owner
5. Keep on-call incidents within agreed SLOs

KR1 Deep Dive: Reducing Hotfix Deployments

Definition:

Hotfix = any out-of-band production deployment triggered by incidents, data corruption, severe

regressions, or broken background jobs.

Execution Discipline:

- Classify the last 30–60 hotfixes and eliminate the top 2–3 recurring causes
- Enforce performance review on hot-path changes (compare P95 before/after merges)
- Enforce Celery discipline: idempotent tasks, bounded retries, task time limits; alert on retry rate
- Enforce safe migrations: backward-compatible schema changes, separated schema/data/code, mandatory rollback plans
- Require a PR checklist covering performance impact, migration safety, Celery safety, and rollback
- Every hotfix must result in one permanent prevention (test, alert, or guardrail)

Success Indicators

Your OKRs are working if:

- Metrics are consistently tracked and visible
- Every team member knows which team OKRs they impact
- Individual OKRs clearly ladder up to team objectives
- You're making data-driven decisions, not fighting fires

Warning Signs:

- Celery queues pile up
- Django endpoints remain slow
- Team is constantly firefighting instead of improving

Metrics & Tracking

Primary Tools:

- **Grafana** - Dashboards for API response times, Celery queue depth, infrastructure metrics, database performance, and autoscaling triggers
- **Sentry** - Error tracking, 5xx monitoring, task failure alerts, and incident management

Dashboard Requirements:

- Each Team OKR must have a dedicated Grafana dashboard with KR progress
- Sentry alerts configured for all P1/P2 error thresholds
- Weekly metrics snapshots exported for OKR review meetings

Last Updated: January 30, 2026

Next Review: Weekly (Metrics) | Monthly (Adjustments) | Quarterly (Full Review)